PW 2: DATA WAREHOUSE REPORT

Data Models in Database Systems, 2025

Flight Operations And Analytics

Group Members:

- Didem Nur Mutlu, 231ADB286
- Fakhri Maharramzade, 231ADB135

Introduction

This report presents the design and implementation of a data warehouse for the **Flight Operations & Analytics** domain. The scenario describes an airline that operates a fleet of aircraft on multiple routes connecting airports across several countries. It sells tickets to passengers, tracks both scheduled and actual departure/arrival times, and wants to analyse route profitability, on-time performance, and passenger load factor. In addition, management is interested in seasonal patterns in ticket sales and the performance of different aircraft models.

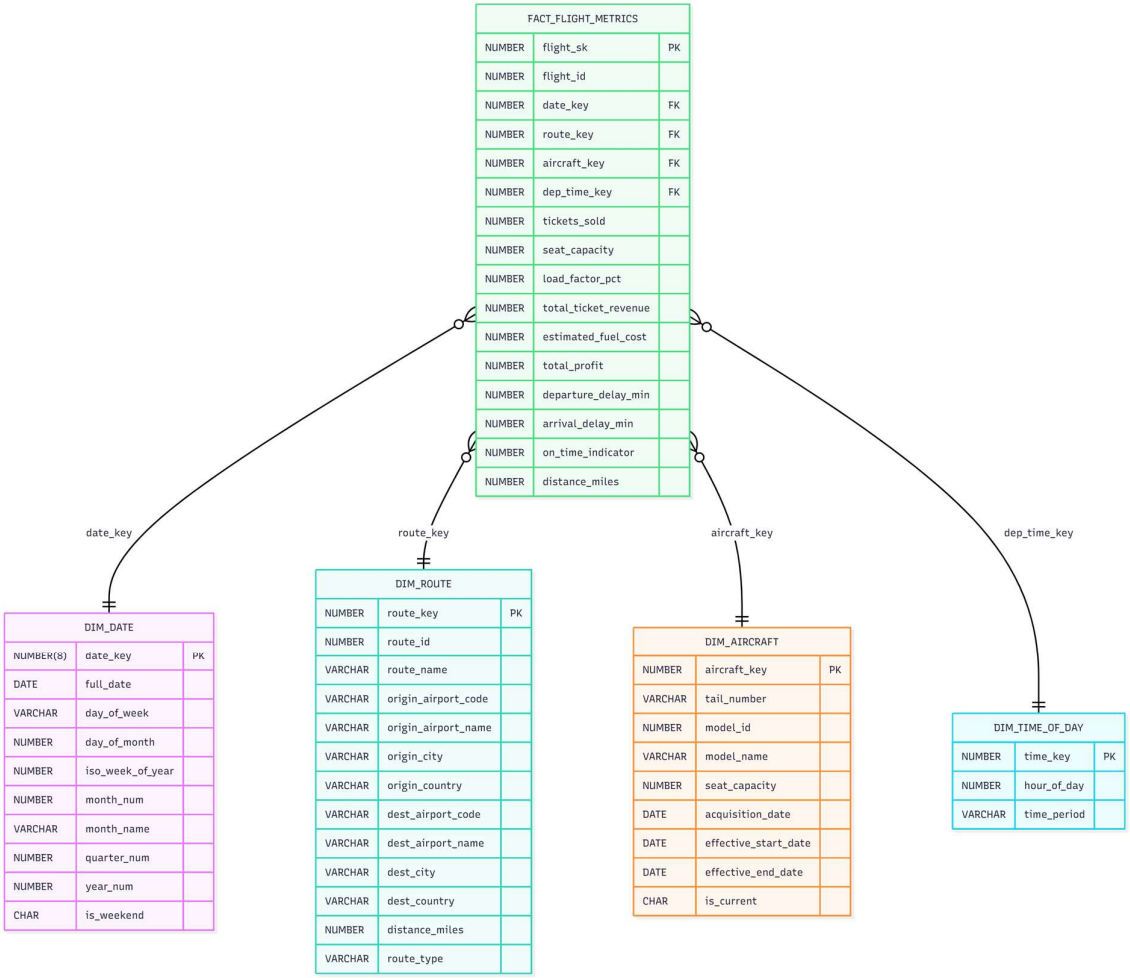
The operational system for this airline already exists as a normalized OLTP schema with tables for airports, routes, aircraft models, individual aircraft (tail numbers), flights, passengers, and tickets. This schema is well suited for day-to-day operations such as booking tickets or updating flight details, but it is not optimized for complex analytical queries. To answer questions such as “Which routes are our most profitable?”, “Which aircraft models are most prone to delays?”, or “How does average delay vary by time of day and airport?”, we need a separate data warehouse designed specifically for analytics.

1. Task 1: Design the Data Warehouse

This section covers our final data warehouse design and the justifications for our decisions.

1.1 ER Diagram

Below is the Entity-Relationship (ER) diagram for our star schema.



1.2. Design Justification

We chose a **star schema** for several reasons. First, the main business process we want to analyse is clearly defined: individual flights operated by the airline. All the analytics—revenue, profit, delay, load factor, and seasonal patterns—are naturally expressed as aggregations over flights. Representing this process as a single central fact table with surrounding dimension tables simplifies the SQL needed for OLAP-style queries, especially those using ROLLUP, CUBE, and GROUPING SETS. Second, a star schema denormalizes descriptive attributes into dimension tables. This matches how analysts think (in terms of time, route, aircraft, and time-of-day) and reduces the number of joins required at query time compared with a snowflake schema. Third, the schema easily accommodates hierarchies within the date and route dimensions and allows us to prepare one dimension (DIM_AIRCRAFT) as a Slowly Changing Dimension Type 2 to keep historical aircraft configurations.

- **Fact Table: FACT_FLIGHT_METRICS**

  Grain: A single row in FACT_FLIGHT_METRICS represents one completed flight leg. In other words, each instance of a flight (unique Flight_ID from the OLTP system) corresponds to one fact row. At this grain, we can attach all relevant measures (delay, revenue, fuel cost, load factor, profit) and analyse them by time, route, aircraft, and departure time of day.

- **Keys**
  - Surrogate primary key: flight_sk (NUMBER, identity).
  - Business identifier: flight_id (NUMBER) given from the OLTP flights table.
  - Foreign Keys:
    - date_key → DIM_DATE(date_key) (scheduled departure date).
    - route_key → DIM_ROUTE(route_key) (origin–destination route).
    - aircraft_key → DIM_AIRCRAFT(aircraft_key) (tail number and model).
    - dep_time_key → DIM_TIME_OF_DAY(time_key) (scheduled departure hour).
- **Measures:**
  - tickets_sold – number of tickets issued for the flight (passengers boarded).
  - seat_capacity – number of seats available on the aircraft for that flight.
  - load_factor_pct – passenger load factor (tickets_sold / seat_capacity * 100).
  - total_ticket_revenue – total ticket revenue for the flight in monetary units.
  - estimated_fuel_cost – estimated fuel and operating cost for the flight, calculated from distance, seat capacity, and aircraft model.
  - total_profit – approximate profit for the flight (total_ticket_revenue - estimated_fuel_cost).
  - departure_delay_min – departure delay in minutes (actual minus scheduled).
  - arrival_delay_min – arrival delay in minutes.
  - on_time_indicator – 1 if arrival delay ≤ 15 minutes, otherwise 0 (for computing on-time performance).
  - distance_miles – distance of the route in miles (denormalized from the route dimension).

These measures directly support the business questions from the domain description: route profitability (revenue, fuel cost, profit), on-time performance (delays and on_time_indicator), load factor per flight, and route performance over time.

Dimension Tables

- **DIM_DATE**: The date dimension provides calendar attributes so that we can analyse flights by day, month, quarter, and year, and observe seasonal patterns in ticket sales and delays. It contains:
    - *Key Attributes:*
        - *date_key (NUMBER(8), primary key in YYYYMMDD format)*
        - full_date (DATE)
        - day_of_week (MONDAY, TUESDAY, …)
        - day_of_month (1–31)
        - iso_week_of_year (1–53)
        - month_num (1–12) and month_name (JAN, FEB, …)
        - quarter_num (1–4)
        - year_num (e.g., 2025)
        - is_weekend (Y/N)
    - *The main hierarchy is **day → month → quarter → year**, which we use in ROLLUP and GROUPING SETS queries.*

- **DIM_TIME_OF_DAY**: This dimension categorizes flights by scheduled departure time. It contains 24 rows, one per hour of the day:
    - *Key Attributes:*
        - *time_key – numeric key from 0 to 23 (primary key).*
        - *hour_of_day – the same number, 0–23.*
        - *time_period – broader category such as Night (0–4), Morning (5–10), Afternoon (11–16), and Evening (17–23).*
    - *This dimension supports questions such as "How does average delay change by time of day?" and is also used in one of the PL/SQL-based analytical queries.*

- **DIM_ROUTE**: The route dimension describes each origin–destination pair in a denormalized way, combining data from the operational `Routes` and `Airports` tables. For each route we store:
    - *Key Attributes:*

- *route_key (primary key, surrogate).*
- *route_id (business key from the OLTP system).*
- *route_name (for example, RIX-FRA).*
- *origin_airport_code, origin_airport_name, origin_city, origin_country.*
- *dest_airport_code, dest_airport_name, dest_city, dest_country.*
- *distance_miles.*
- *route_type – Domestic if origin and destination are in the same country, otherwise International.*
- *Because the origin and destination airport attributes are stored directly in DIM_ROUTE, the schema remains a star rather than a snowflake. We can answer questions "by airport" (e.g., average delay by origin airport or by origin city) simply by grouping on the appropriate attributes of DIM_ROUTE.*

STAGING TABLES

The practical work requires dimension and fact tables to be populated using INSERT ... SELECT statements. Because we only have a conceptual OLTP schema, we simulate it by creating **staging tables** that mirror the operational structure: STG_AIRPORTS, STG_AIRCRAFT_MODELS, STG_FLEET_AIRCRAFT, STG_ROUTES, STG_FLIGHTS, STG_PASSENGERS, and STG_TICKETS.

We insert realistic sample data into the staging tables using INSERT ... VALUES. Then a dedicated ETL procedure, PR_LOAD_FLIGHT_DW, uses INSERT ... SELECT from the staging tables to populate all dimension tables and the fact table. Staging tables are not part of the warehouse ER diagram; they exist solely to simulate the source system and to make the ETL process reproducible.

DIM_AIRCRAFT (Slowly Changing Dimension Type 2).

The aircraft dimension is designed as an SCD Type 2 so that historical changes in aircraft configuration (such as seat reconfigurations) can be tracked over time. Rows are based on tail numbers, and new rows are created when relevant attributes change.

Attributes:

- aircraft_key (NUMBER, surrogate PK).
- tail_number (e.g., YL-ABC) – business key.
- model_id – numeric ID from Aircraft_Models.
- model_name – e.g., Airbus A220-300.

- seat_capacity – number of seats on this aircraft configuration.
- acquisition_date – when the aircraft joined the fleet.
- effective_start_date, effective_end_date – validity period for this configuration.
- is_current – 'Y' for current configuration, 'N' for historical records.

When seat capacity or model changes, the previous row's effective_end_date is set and is_current is updated to 'N', and a new row is inserted with the updated attributes. FACT_FLIGHT_METRICS stores aircraft_key, so each historical flight remains associated with the correct aircraft configuration valid at that time.

2. Task 2: Implement the Warehouse (ETL)

- **CLEANUP: DROP OLD OBJECTS IF THEY EXIST**

```
SET DEFINE OFF;


------------------------------------------------------------
-- 0. CLEANUP: DROP OLD OBJECTS IF THEY EXIST
------------------------------------------------------------


-- Fact & aggregate tables
BEGIN EXECUTE IMMEDIATE 'DROP TABLE fact_flight_metrics PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE agg_monthly_route_perf PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/


-- Dimensions
BEGIN EXECUTE IMMEDIATE 'DROP TABLE dim_time_of_day PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE dim_aircraft PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
```

```
BEGIN EXECUTE IMMEDIATE 'DROP TABLE dim_route PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE dim_date PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/

-- Staging tables
BEGIN EXECUTE IMMEDIATE 'DROP TABLE stg_tickets PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE stg_passengers PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE stg_flights PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE stg_routes PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE stg_fleet_aircraft PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE stg_aircraft_models PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE stg_airports PURGE';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
```

```
-- Functions
BEGIN EXECUTE IMMEDIATE 'DROP FUNCTION fn_estimated_fuel_cost';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP FUNCTION fn_delay_category';
EXCEPTION WHEN OTHERS THEN NULL; END;
/

-- Procedures
BEGIN EXECUTE IMMEDIATE 'DROP PROCEDURE pr_load_flight_dw';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP PROCEDURE pr_populate_agg_monthly_route';
EXCEPTION WHEN OTHERS THEN NULL; END;
/
```

- **CREATING DIM AND FACT TABLES**

```
------------------------------------------------------------
-- 1. DIMENSION TABLES (STAR SCHEMA)
------------------------------------------------------------

-- Time dimension: calendar
CREATE TABLE dim_date (
  date_key          NUMBER(8)   CONSTRAINT pk_dim_date PRIMARY KEY,  -- YYYYMMDD
  full_date         DATE        NOT NULL,
  day_of_week       VARCHAR2(9) NOT NULL,
  day_of_month      NUMBER(2)   NOT NULL,
  iso_week_of_year  NUMBER(2)   NOT NULL,
  month_num         NUMBER(2)   NOT NULL,
```

```
    month_name        VARCHAR2(9) NOT NULL,
    quarter_num       NUMBER(1)   NOT NULL,
    year_num          NUMBER(4)   NOT NULL,
    is_weekend        CHAR(1)     CHECK (is_weekend IN ('Y','N'))
);

-- Route dimension: denormalized origin/destination info
CREATE TABLE dim_route (
    route_key             NUMBER GENERATED BY DEFAULT AS IDENTITY,
    route_id              NUMBER        NOT NULL,
    route_name            VARCHAR2(20)  NOT NULL,
    origin_airport_code   VARCHAR2(3)   NOT NULL,
    origin_airport_name   VARCHAR2(100),
    origin_city           VARCHAR2(50),
    origin_country        VARCHAR2(50),
    dest_airport_code     VARCHAR2(3)   NOT NULL,
    dest_airport_name     VARCHAR2(100),
    dest_city             VARCHAR2(50),
    dest_country          VARCHAR2(50),
    distance_miles        NUMBER(6),
    route_type            VARCHAR2(20),   -- Domestic / International
    CONSTRAINT pk_dim_route PRIMARY KEY (route_key),
    CONSTRAINT uk_dim_route_id UNIQUE (route_id)
);

-- Aircraft dimension: SCD Type 2 based on tail number
CREATE TABLE dim_aircraft (
    aircraft_key        NUMBER GENERATED BY DEFAULT AS IDENTITY,
    tail_number         VARCHAR2(10)  NOT NULL,
    model_id            NUMBER        NOT NULL,
```

```
    model_name          VARCHAR2(50)  NOT NULL,
    seat_capacity       NUMBER(4)     NOT NULL,
    acquisition_date    DATE,
    effective_start_date DATE         NOT NULL,
    effective_end_date   DATE,
    is_current          CHAR(1)       DEFAULT 'Y'
                                      CHECK (is_current IN ('Y','N')),
    CONSTRAINT pk_dim_aircraft PRIMARY KEY (aircraft_key),
    CONSTRAINT uk_dim_aircraft UNIQUE (tail_number, effective_start_date)
);

-- Time-of-day dimension
CREATE TABLE dim_time_of_day (
    time_key     NUMBER(2)    CONSTRAINT pk_dim_time PRIMARY KEY,
    hour_of_day   NUMBER(2)      NOT NULL CHECK (hour_of_day BETWEEN 0
AND 23),
    time_period  VARCHAR2(20) NOT NULL
);
------------------------------------------------------------
-- 3. STAGING TABLES (SIMULATED OLTP SOURCE)
------------------------------------------------------------

CREATE TABLE stg_airports (
    airport_code  VARCHAR2(3),
    airport_name  VARCHAR2(100),
    city          VARCHAR2(50),
    country       VARCHAR2(50)
);

CREATE TABLE stg_aircraft_models (
    model_id      NUMBER,
```

```
    model_name    VARCHAR2(50),
    seat_capacity NUMBER
);

CREATE TABLE stg_fleet_aircraft (
    tail_number     VARCHAR2(10),
    model_id        NUMBER,
    acquisition_date DATE
);

CREATE TABLE stg_routes (
    route_id          NUMBER,
    origin_airport    VARCHAR2(3),
    destination_airport VARCHAR2(3),
    distance_miles    NUMBER
);

CREATE TABLE stg_flights (
    flight_id           NUMBER,
    route_id            NUMBER,
    tail_number         VARCHAR2(10),
    scheduled_departure DATE,
    scheduled_arrival   DATE,
    actual_departure    DATE,
    actual_arrival      DATE
);

CREATE TABLE stg_passengers (
    passenger_id   NUMBER,
    passenger_name VARCHAR2(150),
```

```
    email          VARCHAR2(100)
);

CREATE TABLE stg_tickets (
    ticket_number  VARCHAR2(20),
    flight_id      NUMBER,
    passenger_id   NUMBER,
    price          NUMBER(10,2),
    seat_number    VARCHAR2(10)
);
```

- **PL/SQL Population Procedure:**

```
-------------------------------------------------------------
-- 4. POPULATE STAGING TABLES WITH SAMPLE DATA
-------------------------------------------------------------

-- Airports
INSERT INTO stg_airports VALUES ('RIX', 'Riga International', 'Riga', 'Latvia');
INSERT INTO stg_airports VALUES ('VNO', 'Vilnius International', 'Vilnius', 'Lithuania');
INSERT INTO stg_airports VALUES ('TLL', 'Tallinn Airport', 'Tallinn', 'Estonia');
INSERT INTO stg_airports VALUES ('FRA', 'Frankfurt Main', 'Frankfurt','Germany');
INSERT INTO stg_airports VALUES ('LHR', 'Heathrow', 'London', 'United Kingdom');
INSERT INTO stg_airports VALUES ('DXB', 'Dubai International', 'Dubai', 'UAE');

-- Aircraft models
INSERT INTO stg_aircraft_models VALUES (100, 'Airbus A220-300', 145);
INSERT INTO stg_aircraft_models VALUES (101, 'Airbus A320neo', 180);
INSERT INTO stg_aircraft_models VALUES (102, 'Boeing 737-800', 189);
```

```
INSERT INTO stg_aircraft_models VALUES (103, 'Boeing 787-9',    290);
INSERT INTO stg_aircraft_models VALUES (104, 'ATR 72-600',       70);

-- Fleet aircraft (tail numbers)
INSERT INTO stg_fleet_aircraft VALUES ('YL-ABC', 100, DATE '2022-01-10');
INSERT INTO stg_fleet_aircraft VALUES ('YL-DEF', 101, DATE '2021-05-15');
INSERT INTO stg_fleet_aircraft VALUES ('YL-GHI', 102, DATE '2020-03-20');
INSERT INTO stg_fleet_aircraft VALUES ('YL-JKL', 104, DATE '2019-09-05');
INSERT INTO stg_fleet_aircraft VALUES ('YL-MNO', 101, DATE '2023-02-18');
INSERT INTO stg_fleet_aircraft VALUES ('YL-PQR', 103, DATE '2022-11-30');

-- Routes
INSERT INTO stg_routes VALUES (1, 'RIX', 'VNO', 160);
INSERT INTO stg_routes VALUES (2, 'RIX', 'TLL', 190);
INSERT INTO stg_routes VALUES (3, 'RIX', 'FRA', 840);
INSERT INTO stg_routes VALUES (4, 'RIX', 'LHR', 1020);
INSERT INTO stg_routes VALUES (5, 'VNO', 'FRA', 900);
INSERT INTO stg_routes VALUES (6, 'TLL', 'DXB', 2500);

-- Passengers (a small pool is re-used across flights)
INSERT INTO stg_passengers VALUES (1, 'Alice Johnson',  'alice@example.com');
INSERT INTO stg_passengers VALUES (2, 'Bob Smith',      'bob@example.com');
INSERT INTO stg_passengers VALUES (3, 'Carlos Garcia',  'carlos@example.com');
INSERT INTO stg_passengers VALUES (4, 'Diana Novak',    'diana@example.com');
INSERT INTO stg_passengers VALUES (5, 'Eva Schneider',  'eva@example.com');
INSERT INTO stg_passengers VALUES (6, 'Farid Hussein',  'farid@example.com');

-- Flights (~60 rows with varying routes, aircraft, and times)
DECLARE
   v_base_date DATE := DATE '2025-01-01';
```

```
    v_sched_dep DATE;
    v_sched_arr DATE;
    v_route_id  NUMBER;
    v_tail      VARCHAR2(10);
    v_block_hrs NUMBER;
BEGIN
    FOR i IN 1..60 LOOP
        v_route_id := CASE MOD(i,6)
                        WHEN 1 THEN 1
                        WHEN 2 THEN 2
                        WHEN 3 THEN 3
                        WHEN 4 THEN 4
                        WHEN 5 THEN 5
                        ELSE 6
                      END;

        v_tail := CASE MOD(i,6)
                      WHEN 1 THEN 'YL-ABC'
                      WHEN 2 THEN 'YL-DEF'
                      WHEN 3 THEN 'YL-GHI'
                      WHEN 4 THEN 'YL-JKL'
                      WHEN 5 THEN 'YL-MNO'
                      ELSE 'YL-PQR'
                    END;

        -- Schedule flights on successive days at 6, 12, 18, and 22 o'clock
        v_sched_dep := v_base_date + TRUNC((i-1) / 4)
                       + (CASE MOD(i,4)
                            WHEN 0 THEN 6
                            WHEN 1 THEN 12
```

```
                WHEN 2 THEN 18
                ELSE 22
            END)/24;

-- Block time in hours depending on route length
v_block_hrs := CASE v_route_id
            WHEN 1 THEN 0.8
            WHEN 2 THEN 0.8
            WHEN 3 THEN 2.0
            WHEN 4 THEN 2.3
            WHEN 5 THEN 2.0
            ELSE 5.5
        END;

v_sched_arr := v_sched_dep + v_block_hrs/24;

INSERT INTO stg_flights (
   flight_id, route_id, tail_number,
   scheduled_departure, scheduled_arrival,
   actual_departure, actual_arrival
)
VALUES (
   i,
   v_route_id,
   v_tail,
   v_sched_dep,
   v_sched_arr,
   -- Actual times: introduce random early/late variation
   v_sched_dep + (DBMS_RANDOM.VALUE(-0.15, 0.35))/24,
   v_sched_arr + (DBMS_RANDOM.VALUE(-0.20, 0.60))/24
```

```
        );
    END LOOP;
    COMMIT;
END;
/
SHOW ERRORS;

-- Tickets: generate between 40 and seat_capacity passengers per flight
DECLARE
    v_price_base NUMBER;
BEGIN
    FOR f IN (SELECT flight_id, route_id FROM stg_flights) LOOP
        v_price_base := 80 + 10 * f.route_id; -- simple pricing by route

        FOR p IN 1..DBMS_RANDOM.VALUE(40, 190) LOOP
            INSERT INTO stg_tickets (
                ticket_number, flight_id, passenger_id, price, seat_number
            )
            VALUES (
                'T' || f.flight_id || '-' || p,
                f.flight_id,
                MOD(p,6) + 1,
                v_price_base + DBMS_RANDOM.VALUE(-15, 25),
                'S' || LPAD(p,3,'0')
            );
        END LOOP;
    END LOOP;
    COMMIT;
END;
/
```

```
SHOW ERRORS;
CREATE OR REPLACE PROCEDURE pr_load_flight_dw AS
BEGIN
  ------------------------------------------------------------
  -- Load DIM_DATE: full year 2025 (can be extended)
  ------------------------------------------------------------
  EXECUTE IMMEDIATE 'TRUNCATE TABLE dim_date';

  INSERT INTO dim_date (
    date_key, full_date, day_of_week, day_of_month,
    iso_week_of_year, month_num, month_name,
    quarter_num, year_num, is_weekend
  )
  SELECT
    TO_NUMBER(TO_CHAR(d, 'YYYYMMDD')) AS date_key,
    d                                 AS full_date,
    TO_CHAR(d, 'DAY', 'NLS_DATE_LANGUAGE=English') AS day_of_week,
    TO_NUMBER(TO_CHAR(d, 'DD'))     AS day_of_month,
    TO_NUMBER(TO_CHAR(d, 'IW'))     AS iso_week_of_year,
    TO_NUMBER(TO_CHAR(d, 'MM'))     AS month_num,
    TO_CHAR(d, 'MON', 'NLS_DATE_LANGUAGE=English') AS month_name,
    TO_NUMBER(TO_CHAR(d, 'Q'))      AS quarter_num,
    TO_NUMBER(TO_CHAR(d, 'YYYY'))   AS year_num,
    CASE
      WHEN TO_CHAR(d, 'D', 'NLS_TERRITORY=AMERICA') IN ('1','7')
      THEN 'Y' ELSE 'N'
    END AS is_weekend
  FROM (
    SELECT DATE '2025-01-01' + LEVEL - 1 AS d
    FROM dual
```

```
    CONNECT BY LEVEL <= 365
);

-------------------------------------------------------------
-- Load DIM_ROUTE by joining routes with airports
-------------------------------------------------------------
EXECUTE IMMEDIATE 'TRUNCATE TABLE dim_route';

INSERT INTO dim_route (
    route_id, route_name,
    origin_airport_code, origin_airport_name, origin_city, origin_country,
    dest_airport_code, dest_airport_name, dest_city, dest_country,
    distance_miles, route_type
)
SELECT
    r.route_id,
    r.origin_airport || '-' || r.destination_airport AS route_name,
    ao.airport_code     AS origin_airport_code,
    ao.airport_name     AS origin_airport_name,
    ao.city             AS origin_city,
    ao.country          AS origin_country,
    ad.airport_code     AS dest_airport_code,
    ad.airport_name     AS dest_airport_name,
    ad.city             AS dest_city,
    ad.country          AS dest_country,
    r.distance_miles,
    CASE
        WHEN ao.country = ad.country THEN 'Domestic'
        ELSE 'International'
    END AS route_type
```

```
FROM stg_routes r
    JOIN stg_airports ao ON r.origin_airport = ao.airport_code
    JOIN stg_airports ad ON r.destination_airport = ad.airport_code;

------------------------------------------------------------
-- Load DIM_AIRCRAFT (SCD Type 2 - initial snapshot)
------------------------------------------------------------
EXECUTE IMMEDIATE 'TRUNCATE TABLE dim_aircraft';

INSERT INTO dim_aircraft (
   tail_number, model_id, model_name,
   seat_capacity, acquisition_date,
   effective_start_date, effective_end_date, is_current
)
SELECT
   fa.tail_number,
   am.model_id,
   am.model_name,
   am.seat_capacity,
   fa.acquisition_date,
   fa.acquisition_date       AS effective_start_date,
   NULL                      AS effective_end_date,
   'Y'                       AS is_current
FROM stg_fleet_aircraft fa
    JOIN stg_aircraft_models am
      ON fa.model_id = am.model_id;

------------------------------------------------------------
-- Load DIM_TIME_OF_DAY (24 rows)
------------------------------------------------------------
```

```
EXECUTE IMMEDIATE 'TRUNCATE TABLE dim_time_of_day';

INSERT INTO dim_time_of_day (time_key, hour_of_day, time_period)
SELECT
   h AS time_key,
   h AS hour_of_day,
   CASE
      WHEN h BETWEEN 5 AND 10  THEN 'Morning'
      WHEN h BETWEEN 11 AND 16 THEN 'Afternoon'
      WHEN h BETWEEN 17 AND 21 THEN 'Evening'
      ELSE 'Night'
   END AS time_period
FROM (
   SELECT LEVEL-1 AS h
   FROM dual
   CONNECT BY LEVEL <= 24
);

-------------------------------------------------------------
-- Load FACT_FLIGHT_METRICS
-------------------------------------------------------------
EXECUTE IMMEDIATE 'TRUNCATE TABLE fact_flight_metrics';

INSERT INTO fact_flight_metrics (
   flight_id,
   date_key, route_key, aircraft_key, dep_time_key,
   tickets_sold, seat_capacity, load_factor_pct,
   total_ticket_revenue, estimated_fuel_cost, total_profit,
   departure_delay_min, arrival_delay_min, on_time_indicator,
   distance_miles
```

```
)
WITH tkt AS (
   SELECT
      flight_id,
      COUNT(*)            AS tickets_sold,
      SUM(price)          AS total_revenue
   FROM stg_tickets
   GROUP BY flight_id
)
SELECT
   f.flight_id,
   dd.date_key,
   dr.route_key,
   da.aircraft_key,
   td.time_key,
   NVL(t.tickets_sold,0)                    AS tickets_sold,
   da.seat_capacity,
   ROUND( NVL(t.tickets_sold,0) / da.seat_capacity * 100, 2 ) AS load_factor_pct,
   NVL(t.total_revenue,0)                    AS total_ticket_revenue,
   fn_estimated_fuel_cost(r.distance_miles, da.seat_capacity, da.model_name)
                                    AS estimated_fuel_cost,
   NVL(t.total_revenue,0)
     - fn_estimated_fuel_cost(r.distance_miles, da.seat_capacity, da.model_name)
                                    AS total_profit,
   ROUND( (f.actual_departure - f.scheduled_departure) * 24 * 60, 1 )
                                    AS departure_delay_min,
   ROUND( (f.actual_arrival   - f.scheduled_arrival)   * 24 * 60, 1 )
                                    AS arrival_delay_min,
   CASE
      WHEN (f.actual_arrival - f.scheduled_arrival) * 24 * 60 <= 15
```

```
        THEN 1 ELSE 0
      END                                  AS on_time_indicator,
      r.distance_miles
    FROM stg_flights f
      JOIN stg_routes r          ON f.route_id   = r.route_id
      JOIN dim_route dr          ON dr.route_id  = r.route_id
      JOIN stg_fleet_aircraft fa ON f.tail_number = fa.tail_number
      JOIN dim_aircraft da
        ON da.tail_number = fa.tail_number AND da.is_current = 'Y'
      JOIN stg_aircraft_models am ON fa.model_id = am.model_id
      LEFT JOIN tkt t            ON t.flight_id  = f.flight_id
      JOIN dim_date dd
        ON dd.full_date = TRUNC(f.scheduled_departure)
      JOIN dim_time_of_day td
        ON                                td.hour_of_day                                =
TO_NUMBER(TO_CHAR(f.scheduled_departure,'HH24'));

    COMMIT;
END;
/
SHOW ERRORS;
BEGIN
    pr_load_flight_dw;
END;
/
```

3. Task 3: Advanced Analytical Queries

3.1 ROLLUP/CUBE Queries

Query 1 – ROLLUP: Revenue, Profit, and Load Factor by Year, Quarter, and Route

- **Business Question:** How much ticket revenue and profit do we earn per route, quarter, and year, and what is the average load factor? We also want yearly totals and a grand total across all years and routes.
- **SQL Query:**

```
SELECT
   dd.year_num,
   dd.quarter_num,
   dr.route_name,
   SUM(ff.total_ticket_revenue) AS total_revenue,
   SUM(ff.total_profit)        AS total_profit,
   ROUND(AVG(ff.load_factor_pct),2) AS avg_load_factor,
   GROUPING(dd.year_num)       AS grp_year,
   GROUPING(dd.quarter_num)    AS grp_quarter,
   GROUPING(dr.route_name)     AS grp_route
FROM fact_flight_metrics ff
    JOIN dim_date  dd ON ff.date_key  = dd.date_key
    JOIN dim_route dr ON ff.route_key = dr.route_key
GROUP BY ROLLUP (dd.year_num, dd.quarter_num, dr.route_name)
ORDER BY dd.year_num, dd.quarter_num, dr.route_name;
```

- **Technical Explanation:** ROLLUP (year_num, quarter_num, route_name) produces aggregations at four levels: (year, quarter, route), (year, quarter), (year), and the overall total. The GROUPING function marks which columns are being rolled up so that the application or report can distinguish detail rows from subtotal and grand total rows.

Query 2 – CUBE: Delay and On-Time Rate by Origin Country and Aircraft Model

- **Business Question:** What is the average arrival delay and on-time percentage by **origin country** and **aircraft model**, including all subtotals by country only, model only, and overall?
- **SQL Query:**

```
SELECT
   dr.origin_country,
   da.model_name,
   ROUND(AVG(ff.arrival_delay_min),1)         AS avg_arrival_delay_min,
   ROUND(AVG(ff.on_time_indicator)*100,2)     AS on_time_pct
FROM fact_flight_metrics ff
    JOIN dim_route    dr ON ff.route_key    = dr.route_key
    JOIN dim_aircraft da ON ff.aircraft_key = da.aircraft_key
GROUP BY CUBE (dr.origin_country, da.model_name)
ORDER BY dr.origin_country, da.model_name;
```

Technical Explanation: CUBE(origin_country, model_name) creates aggregates for every combination of the two dimensions plus all their subtotals and the grand total. The query returns results for specific (country, model), for each country across all

models, for each model across all countries, and for the entire dataset. This allows us to quickly see which aircraft models and which countries are most prone to delays.

Query 3 – GROUPING SETS: Monthly Revenue and Load Factor by Route and Route Type

- **Business Question:** How do revenue and load factor develop over time, both at the detailed **route** level and at a higher **route type** level (Domestic vs International), as well as overall per month?
- **SQL Query:**

```
 SELECT
    dd.year_num,
    dd.month_num,
    dr.route_type,
    dr.route_name,
    SUM(ff.total_ticket_revenue)       AS total_revenue,
    ROUND(AVG(ff.load_factor_pct), 2)  AS avg_load_factor
FROM fact_flight_metrics ff
     JOIN dim_date  dd ON ff.date_key  = dd.date_key
     JOIN dim_route dr ON ff.route_key = dr.route_key
GROUP BY GROUPING SETS (
    (dd.year_num, dd.month_num, dr.route_type, dr.route_name),
    (dd.year_num, dd.month_num, dr.route_type),
    (dd.year_num, dd.month_num),
    ()
)
ORDER BY dd.year_num, dd.month_num, dr.route_type, dr.route_name;
```

Technical Explanation: GROUPING SETS explicitly lists the desired aggregation levels: per (year, month, route_type, route), per (year, month, route_type), per (year, month), and a grand total. This is more flexible than ROLLUP because we only calculate the rollups we actually need and avoid meaningless intermediate levels.

Query 4 – ROLLUP with PL/SQL Function: Flights by Delay Category and Year

- **Business Question:** How many flights fall into each delay category (EARLY, ON_TIME, LATE, VERY_LATE) per year, and what is the average arrival delay in each category, including yearly totals and the overall total?
- **SQL Query:**

```
SELECT
    dd.year_num,
    fn_delay_category(ff.arrival_delay_min) AS delay_category,
    COUNT(*)                                AS flight_count,
    ROUND(AVG(ff.arrival_delay_min),1)      AS avg_arrival_delay,
    GROUPING(dd.year_num)                   AS grp_year,
```

```
    GROUPING(fn_delay_category(ff.arrival_delay_min)) AS grp_cat
FROM fact_flight_metrics ff
    JOIN dim_date dd ON ff.date_key = dd.date_key
GROUP BY ROLLUP (dd.year_num, fn_delay_category(ff.arrival_delay_min))
ORDER BY dd.year_num, delay_category;
```

Technical Explanation: This query uses the PL/SQL function fn_delay_category inside both the SELECT and GROUP BY clauses. Each flight's numeric arrival delay is converted into a categorical label, and ROLLUP generates counts and averages per (year, category), per year, and overall. GROUPING flags are available to distinguish totals from detail rows.

3.2 PL/SQL Functions

Function 1: Estimated Fuel Cost per Flight

```
CREATE OR REPLACE FUNCTION fn_estimated_fuel_cost (
   p_distance_miles IN NUMBER,
   p_seat_capacity  IN NUMBER,
   p_model_name     IN VARCHAR2
) RETURN NUMBER IS
   v_base_rate_per_mile NUMBER;
   v_capacity_factor     NUMBER;
BEGIN
   IF p_distance_miles IS NULL OR p_distance_miles <= 0 THEN
      RETURN 0;
   END IF;

   -- Simplified model-based cost per mile
   IF p_model_name LIKE 'ATR%' THEN
      v_base_rate_per_mile := 3.0;      -- turboprops are cheaper
   ELSIF p_model_name LIKE 'Airbus A220%' THEN
      v_base_rate_per_mile := 4.0;
   ELSIF p_model_name LIKE 'Airbus A320%' THEN
      v_base_rate_per_mile := 4.5;
```

```
    ELSIF p_model_name LIKE 'Boeing 737%' THEN
        v_base_rate_per_mile := 4.7;
    ELSE
        v_base_rate_per_mile := 5.0;      -- widebodies etc.
    END IF;

    -- Adjust cost slightly for seat capacity (larger aircraft burn more fuel)
    v_capacity_factor := 1 + NVL(p_seat_capacity,150) / 600;

    RETURN ROUND(p_distance_miles * v_base_rate_per_mile * v_capacity_factor, 2);
END;
/
SHOW ERRORS;
SELECT
    da.model_name,
    dr.route_type,
    ROUND(
        AVG(
            fn_estimated_fuel_cost(ff.distance_miles,
                                   ff.seat_capacity,
                                   da.model_name)
            / NULLIF(ff.distance_miles,0)
        ),
        2
    ) AS avg_fuel_cost_per_mile
FROM fact_flight_metrics ff
     JOIN dim_route    dr ON ff.route_key    = dr.route_key
     JOIN dim_aircraft da ON ff.aircraft_key = da.aircraft_key
GROUP BY da.model_name, dr.route_type
ORDER BY da.model_name, dr.route_type;
```

- **Explanation:** This function estimates fuel and operating cost for a flight based on route distance, aircraft seat capacity, and model family. It is used during ETL to populate the estimated_fuel_cost column and also in one of the analytical queries.

Function 2: Delay Category

```
CREATE OR REPLACE FUNCTION fn_delay_category (
   p_arrival_delay_min IN NUMBER
) RETURN VARCHAR2 IS
BEGIN
   IF p_arrival_delay_min IS NULL THEN
      RETURN 'UNKNOWN';
   ELSIF p_arrival_delay_min <= -10 THEN
      RETURN 'EARLY';
   ELSIF p_arrival_delay_min <= 15 THEN
      RETURN 'ON_TIME';
   ELSIF p_arrival_delay_min <= 60 THEN
      RETURN 'LATE';
   ELSE
      RETURN 'VERY_LATE';
   END IF;
END;
/
SHOW ERRORS;
```

- **Explanation:** This function takes arrival delay in minutes and classifies the flight into one of four categories. It will later be used both in SELECT and GROUP BY clauses to provide categorical on-time performance metrics

3.3 Advanced Queries (using Functions, MODEL, etc.)

Query 1: MODEL Clause: What-If Analysis for Fuel Price Increase

- **Business Question: If fuel prices increase by 15%, how would yearly route profitability change?**
-
- **SQL Query:**

```
SELECT
   year_num,
   route_name,
   scenario,
   ROUND(total_revenue,2)    AS total_revenue,
   ROUND(total_fuel_cost,2)  AS total_fuel_cost,
   ROUND(total_profit,2)     AS total_profit
FROM (
   SELECT
      dd.year_num,
      dr.route_name,
      'BASE' AS scenario,
      SUM(ff.total_ticket_revenue) AS total_revenue,
      SUM(ff.estimated_fuel_cost)  AS total_fuel_cost,
      SUM(ff.total_profit)         AS total_profit
   FROM fact_flight_metrics ff
       JOIN dim_date  dd ON ff.date_key  = dd.date_key
       JOIN dim_route dr ON ff.route_key = dr.route_key
   GROUP BY dd.year_num, dr.route_name
)
MODEL
   PARTITION BY (year_num, route_name)
   DIMENSION BY (scenario)
   MEASURES (total_revenue, total_fuel_cost, total_profit)
   RULES UPSERT (
```

```
    -- Create a WHAT-IF scenario with 15% higher fuel cost
    total_revenue['FUEL+15']   = total_revenue['BASE'],
    total_fuel_cost['FUEL+15'] = total_fuel_cost['BASE'] * 1.15,
    total_profit['FUEL+15']    = total_revenue['FUEL+15']
                               - total_fuel_cost['FUEL+15']
  )
ORDER BY year_num, route_name, scenario;
```

- **Technical Explanation:** The inner query aggregates revenue, fuel cost, and profit by year and route, tagging each row with scenario 'BASE'. The MODEL clause partitions the data by (year_num, route_name) and defines scenario as the dimension with three measures. The RULES UPSERT section creates additional rows for scenario 'FUEL+15', where fuel cost is increased by 15% while revenue is unchanged. Profit for the what-if scenario is recomputed accordingly. The result set therefore contains two rows per (year, route): one for base reality and one for the hypothetical fuel price increase.

4. Task 4: Aggregate Table

High-level dashboards for example, a monthly profitability chart per route—should not aggregate directly over millions of fact rows. Instead, we design an aggregate table that precomputes monthly route performance.

- **Justification:** The key business metric identified for pre-aggregation is: Monthly route performance by route and year, including number of flights, total tickets sold, total revenue, total estimated fuel cost, profit, average load factor, average arrival delay, and on-time percentage.

- **DDL for Aggregate Table:**
  - The aggregate table is named AGG_MONTHLY_ROUTE_PERF. This table stores one row per (year, month, route) and aggregates all relevant KPIs.

```
CREATE TABLE agg_monthly_route_perf (
  year_num            NUMBER(4)    NOT NULL,
  month_num           NUMBER(2)    NOT NULL,
  route_key           NUMBER       NOT NULL,
  route_name          VARCHAR2(20) NOT NULL,

  total_flights       NUMBER,
  total_tickets_sold  NUMBER,
  total_revenue       NUMBER(14,2),
```

```
    total_est_fuel_cost   NUMBER(14,2),
    total_profit          NUMBER(14,2),
    avg_load_factor_pct   NUMBER(5,2),
    avg_arrival_delay_min NUMBER(6,2),
    on_time_pct           NUMBER(5,2),

    CONSTRAINT pk_agg_monthly_route PRIMARY KEY (year_num, month_num,
route_key)
);
```

- **Analytical Query on the Aggregate Table**

  A typical dashboard query becomes very simple and fast when run on the aggregate table instead of the detailed fact table. This query returns already-aggregated KPIs for each route per month in 2025.

```
SELECT
    year_num,
    month_num,
    route_name,
    total_flights,
    total_revenue,
    total_profit,
    avg_load_factor_pct,
    on_time_pct
FROM agg_monthly_route_perf
WHERE year_num = 2025
ORDER BY month_num, route_name;
```

- **Population Procedure:**

  We implement a PL/SQL procedure `PR_POPULATE_AGG_MONTHLY_ROUTE` that truncates and repopulates the aggregate table from the fact table.

```
CREATE OR REPLACE PROCEDURE pr_populate_agg_monthly_route AS
BEGIN
   EXECUTE IMMEDIATE 'TRUNCATE TABLE agg_monthly_route_perf';

   INSERT INTO agg_monthly_route_perf (
      year_num, month_num, route_key, route_name,
      total_flights, total_tickets_sold,
      total_revenue, total_est_fuel_cost, total_profit,
      avg_load_factor_pct, avg_arrival_delay_min, on_time_pct
   )
   SELECT
      dd.year_num,
      dd.month_num,
      dr.route_key,
      dr.route_name,
      COUNT(*)                    AS total_flights,
      SUM(ff.tickets_sold)          AS total_tickets_sold,
      SUM(ff.total_ticket_revenue)   AS total_revenue,
      SUM(ff.estimated_fuel_cost)    AS total_est_fuel_cost,
      SUM(ff.total_profit)          AS total_profit,
      ROUND(AVG(ff.load_factor_pct), 2)   AS avg_load_factor_pct,
      ROUND(AVG(ff.arrival_delay_min),2)  AS avg_arrival_delay_min,
      ROUND(AVG(ff.on_time_indicator)*100,2) AS on_time_pct
   FROM fact_flight_metrics ff
       JOIN dim_date  dd ON ff.date_key  = dd.date_key
       JOIN dim_route dr ON ff.route_key = dr.route_key
   GROUP BY dd.year_num, dd.month_num, dr.route_key, dr.route_name;

   COMMIT;
END;
```

```
/

SHOW ERRORS;

BEGIN

   pr_populate_agg_monthly_route;

END;

/
```

- **Performance EXPLAIN PLAN Analysis:**
    - To demonstrate the performance benefit of the aggregate table, we compare the execution plans of two similar queries: one that calculates the metrics directly from FACT_FLIGHT_METRICS and one that reads from AGG_MONTHLY_ROUTE_PERF.
- **Slow query on the fact table**: This plan typically shows a full scan of FACT_FLIGHT_METRICS, joins to DIM_DATE and DIM_ROUTE, and a relatively expensive HASH GROUP BY to compute aggregates.

```
EXPLAIN PLAN FOR
SELECT
   dd.year_num,
   dd.month_num,
   dr.route_name,
   COUNT(*)                    AS total_flights,
   SUM(ff.total_ticket_revenue) AS total_revenue,
   SUM(ff.total_profit)        AS total_profit,
   ROUND(AVG(ff.load_factor_pct),2) AS avg_load_factor_pct,
   ROUND(AVG(ff.on_time_indicator)*100,2) AS on_time_pct
FROM fact_flight_metrics ff
    JOIN dim_date  dd ON ff.date_key  = dd.date_key
    JOIN dim_route dr ON ff.route_key = dr.route_key
GROUP BY dd.year_num, dd.month_num, dr.route_name;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Welcome Page | FLIGHT

Worksheet | Query Builder

```sql
EXPLAIN PLAN FOR
SELECT
    dd.year_num,
    dd.month_num,
    dr.route_name,
    COUNT(*)                      AS total_flights,
    SUM(ff.total_ticket_revenue) AS total_revenue,
    SUM(ff.total_profit)          AS total_profit,
    ROUND(AVG(ff.load_factor_pct),2) AS avg_load_factor_pct,
    ROUND(AVG(ff.on_time_indicator)*100,2) AS on_time_pct
FROM fact_flight_metrics ff
     JOIN dim_date  dd ON ff.date_key  = dd.date_key
     JOIN dim_route dr ON ff.route_key = dr.route_key
GROUP BY dd.year_num, dd.month_num, dr.route_name;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Script Output | Query Result

SQL | All Rows Fetched: 25 in 0.032 seconds

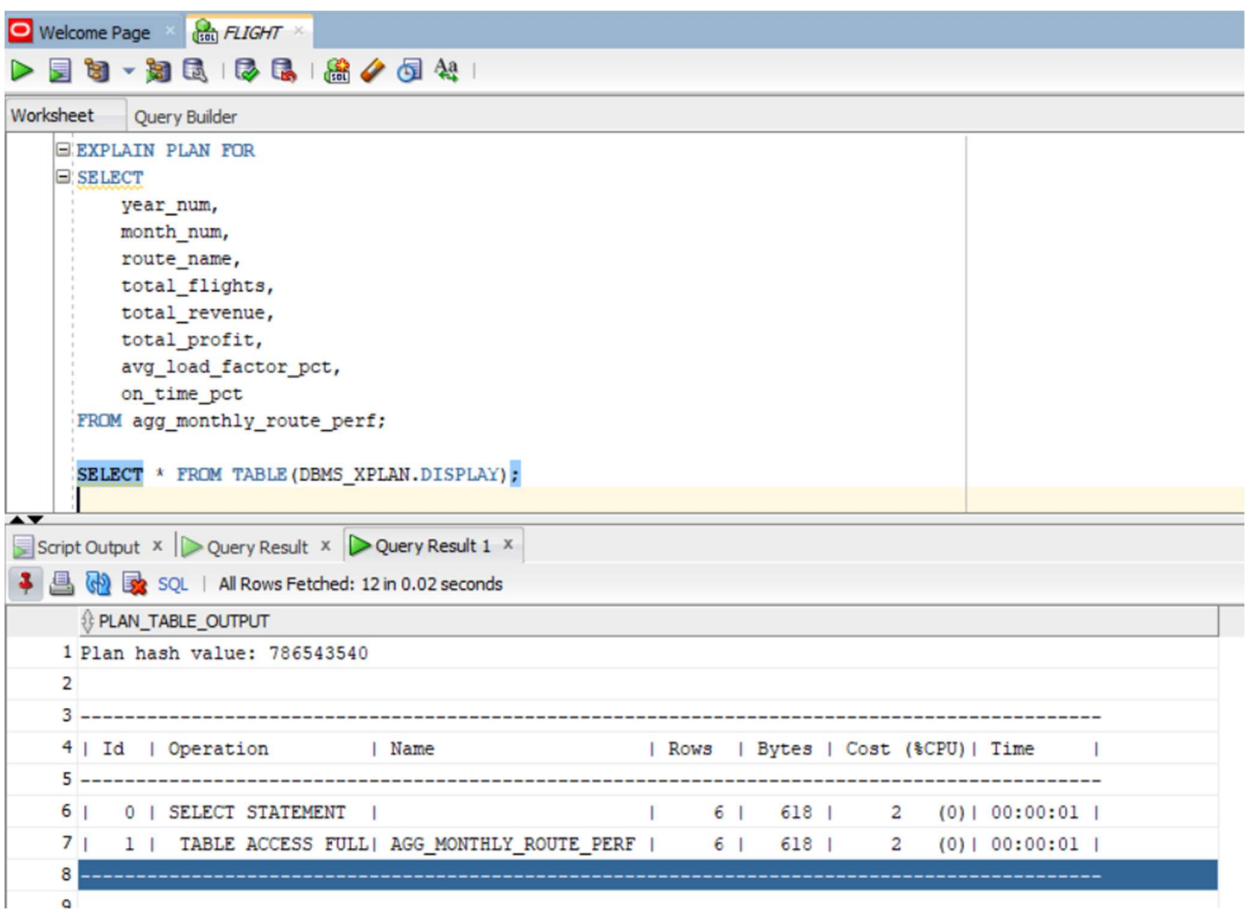
PLAN_TABLE_OUTPUT

```
Plan hash value: 4091862797

-----------------------------------------------------------------------------------------------------
| Id  | Operation                    | Name                | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                     |    60 |  8520 |     6  (17)| 00:00:01 |
|   1 |  HASH GROUP BY               |                     |    60 |  8520 |     6  (17)| 00:00:01 |
|*  2 |   HASH JOIN                  |                     |    60 |  8520 |     5   (0)| 00:00:01 |
|   3 |    NESTED LOOPS              |                     |    60 |  6180 |     2   (0)| 00:00:01 |
|   4 |     NESTED LOOPS             |                     |    60 |  6180 |     2   (0)| 00:00:01 |
|   5 |      TABLE ACCESS FULL       | DIM_ROUTE           |     6 |   150 |     2   (0)| 00:00:01 |
|*  6 |      INDEX RANGE SCAN        | IX_FACT_ROUTE       |    10 |       |     0   (0)| 00:00:01 |
|   7 |     TABLE ACCESS BY INDEX ROWID| FACT_FLIGHT_METRICS |    10 |   780 |     0   (0)| 00:00:01 |
|   8 |    TABLE ACCESS FULL         | DIM_DATE            |   365 | 14235 |     3   (0)| 00:00:01 |
```

- **Fast query on the aggregate table:** The second plan shows a much lower estimated cost, usually involving just a full scan (or index range scan) of AGG_MONTHLY_ROUTE_PERF with no joins and minimal computation. This confirms that pre-aggregation dramatically reduces work for high-level dashboards, especially when the fact table grows to millions of rows.

```
EXPLAIN PLAN FOR
SELECT
   dd.year_num,
   dd.month_num,
   dr.route_name,
   COUNT(*)                    AS total_flights,
   SUM(ff.total_ticket_revenue) AS total_revenue,
   SUM(ff.total_profit)        AS total_profit,
   ROUND(AVG(ff.load_factor_pct),2) AS avg_load_factor_pct,
   ROUND(AVG(ff.on_time_indicator)*100,2) AS on_time_pct
FROM fact_flight_metrics ff
    JOIN dim_date  dd ON ff.date_key  = dd.date_key
    JOIN dim_route dr ON ff.route_key = dr.route_key
GROUP BY dd.year_num, dd.month_num, dr.route_name;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```



5. Demonstration Video Link

A link to our 5-7 minute group video demonstration:

- **Link: https://youtu.be/Ui3q-0mnOVA**

6. Group Work Report

Work Division:

Didem focused on schema design (Task 1). They interpreted the domain description, defined the grain of the fact table, designed the DIM_DATE, DIM_ROUTE, DIM_AIRCRAFT, and DIM_TIME_OF_DAY dimensions, and prepared the ER diagram and implemented the warehouse in Oracle (Task 2). They created all staging and warehouse tables, wrote sample data inserts for the staging area, and

Fakhri implemented the PR_LOAD_FLIGHT_DW ETL procedure and concentrated on analytical queries and the aggregate table (Tasks 3 and 4). They implemented the PL/SQL functions, the ROLLUP/CUBE/GROUPING SETS queries, the MODEL clause scenario, the aggregate table AGG_MONTHLY_ROUTE_PERF, and its population procedure.

Process:

A key early decision was defining the fact table grain as one completed flight leg. Once that was fixed, it became clear which dimensions were needed and which measures to store.

One challenge was balancing realism with simplicity. For example, estimating fuel cost per flight is complex in reality, but for this assignment we needed a transparent formula that could be expressed in PL/SQL. Another difficulty was getting all joins in the ETL procedure correct, especially when mapping staging tables to dimension keys. The group resolved this by testing each part incrementally in SQL Developer and verifying row counts after each step.

Working as a group helped catch errors early. For instance, when the first version of the ROLLUP query produced unexpected subtotal rows, another member diagnosed that the grouping order was wrong. Regular online meetings and sharing scripts through a common repository kept the work coordinated and prevented version conflicts.

7. Conclusions

This practical work guided us through the complete lifecycle of building a data warehouse for an airline's flight operations. Starting from a high-level domain description, we first clarified the main analytical goal: to evaluate route profitability, on-time performance, and load factors over time and across different aircraft models. The critical design decision was to set the grain of the central fact table to a single completed flight leg. Given that grain, we could systematically define the necessary measures (revenue, delay, load factor, fuel cost, profit) and choose dimensions (time, route, aircraft, time of day) that reflect how the business analyses its operations.

Implementing the star schema in Oracle showed the difference between operational and analytical data models. The normalized OLTP-like staging tables captured individual airports, routes, aircraft, flights, passengers, and tickets. The ETL procedure then reshaped this data into denormalized dimensions, and a fact table optimized for queries. Introducing the DIM_AIRCRAFT table as an SCD Type 2 highlighted how data warehouses preserve historical configurations so that past flights can be analysed with the aircraft characteristics that were valid at the time.

The advanced SQL queries demonstrated that many sophisticated analyses can be expressed directly in SQL once the schema is correctly designed. ROLLUP, CUBE, and GROUPING SETS made it straightforward to compute multi-level aggregates for revenue, profit, load factor, delays, and on time performance. The PL/SQL functions encapsulated airline-specific logic: estimating fuel cost from distance and aircraft type and categorizing flights by delay. The MODEL clause added a powerful what-if capability, allowing us to examine the effect of a hypothetical fuel price increase on route profitability without changing any stored data.

Finally, the creation of the AGG_MONTHLY_ROUTE_PERF table illustrated the role of aggregate tables in real-world data warehouses. By comparing EXPLAIN PLAN outputs for similar queries on the fact table and on the aggregate table, we saw how pre-aggregation dramatically reduces query cost and complexity, which is essential for responsive dashboards.